

AUDIT REPORT

September 2026

For

Zynk

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com

Table of Content

Executive Summary	03
Number of Issues per Severity	06
Checked Vulnerabilities	07
Techniques and Methods	08
Types of Severity	10
Types of Issues	12
Severity Matrix	13
Summary of Issues	14
High Severity Issues	16
Medium Severity Issues	19
Low Severity Issues	22
Informational Severity Issues	26
Security Invariants and Non-Negotiable Properties	32
Functional Tests	38
Automated Tests	43
Closing Summary & Disclaimer	44
About QuillAudits	46



Executive Summary

Project Name	Zynk
Protocol Type	DeFi Application, lending/borrowing
Project URL	https://www.zynk.money/
Overview	Zynk is a two-program Solana credit protocol. zynk-core is the settlement and authority layer: it holds the global configuration, the operational vaults (ZOV), the partner deposit vaults (PDV), and the per-order debt record. zynk-orbit is the user-accounting layer: it holds each user's principal ledger, their per-order positions, and the vaults that pay them
Audit Scope	The scope of this audit was to analyze the Zynk smart contracts for quality, security, and correctness.
Source Code link	https://github.com/Zynklabs/zynk-protocol-solana-programs/tree/zynk-core-v4.2+orbit-v2/programs



Contracts in Scope **Zynk Core — programs/zynk-core/src/**

lib.rs	instructions/create_order.rs
contexts.rs	instructions/pull_and_create_order.rs
constants.rs	instructions/record_order.rs
error.rs	instructions/replenish.rs
events.rs	instructions/replenish_and_repay.rs
state.rs	instructions/close_orders.rs
utils.rs	instructions/beneficiaries.rs
instructions/mod.rs	instructions/token_whitelist.rs
instructions/initialize.rs	instructions/governance.rs

Zynk Orbit — programs/zynk-orbit/src/

lib.rs	instructions/repay.rs
contexts.rs	instructions/claim.rs
constants.rs	instructions/disburse.rs
error.rs	instructions/users.rs
events.rs	instructions/withdrawals.rs
state.rs	instructions/cliff.rs
utils.rs	instructions/pledge.rs
instructions/mod.rs	instructions/revoke.rs
instructions/deposit.rs	instructions/cctp.rs
instructions/borrow.rs	



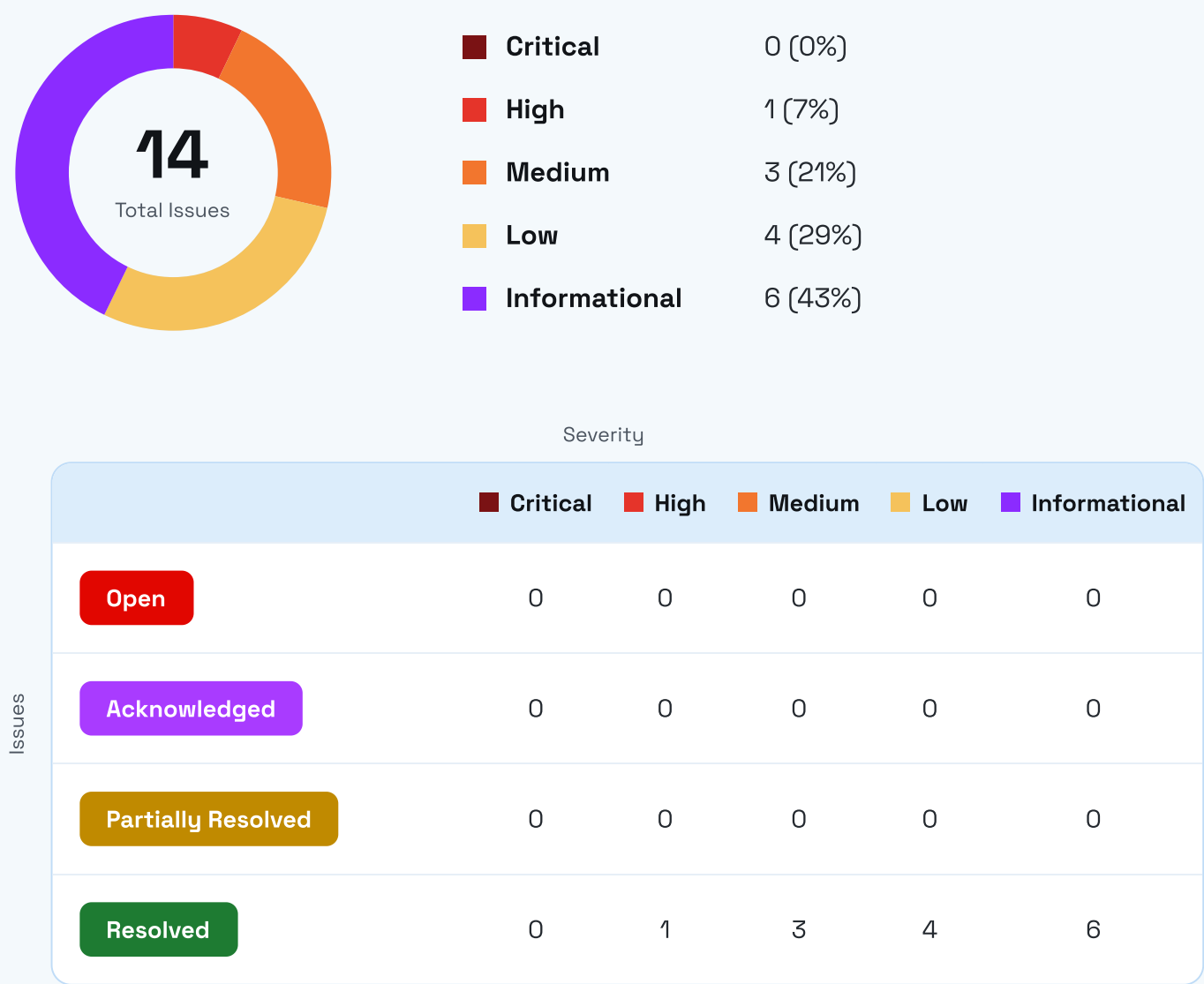
Branch	zynk-core-v4.2+orbit-v2
Commit Hash	21174ea7f986032775ece23bea018ed985e39cd8
Language	Rust
Blockchain	Solana
Method	Manual Analysis, Threat Modelling
Review 1	27th August - 9th September , 2026
Updated Code Received	11th September 2026
Review 2	12th September 2026 - 14th September 2026
Fixed In	c75c7ff

✓ Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Checked Vulnerabilities

We have scanned the solana program for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that we considered:



Signer authorization



Account data matching



Sysvar address checking



Owner checks



Type cosplay



Initialization



Arbitrary cpi



Duplicate mutable accounts



Bump seed canonicalization



PDA Sharing



Incorrect closing accounts



Missing rent exemption checks



Arithmetic overflows/underflows



Numerical precision errors



Solana account confusions



Casting truncation



Insufficient SPL token account verification



Signed invocation of unverified programs



Techniques and Methods

Throughout the audit of Solana Programs, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments match logic and expected behaviour.
- Token distribution and calculations are as per the intended behaviour mentioned in the whitepaper.
- Implementation of various token standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the Solana programs.

Structural Analysis

In this step, we have analysed the design patterns and structure of Solana programs. A thorough check was done to ensure the Solana program is structured in a way that will not result in future problems.

Static Analysis

Static analysis of Solana programs was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of Solana programs.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analysed, and their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behaviour of Solana programs in production. Checks were done to know how much gas gets consumed and the possibilities of optimising code to reduce gas consumption.

Tools and Platforms used for Audit

Manual analysis, threat modelling, functional testing.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.



■ Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.



Severity Matrix

		Impact		
		■ High	■ Medium	■ Low
Likelihood	■ High	Critical	High	Medium
	■ Medium	High	Medium	Low
	■ Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Summary of Issues

No	Issue Title	Severity	Status
1	Un-widened u64 share arithmetic caps a repayable order at 2^{32} base units	High	Resolved
2	Credited amounts are taken from instruction arguments rather than a measured balance delta	Medium	Resolved
3	repay's all-or-nothing settlement gate is unsatisfiable for large position sets and breaks permanently on any divergence	Medium	Resolved
4	The withdrawal amount is u32 in a protocol whose every other principal quantity is u64	Medium	Resolved
5	"ICV custody is consumed first" is documented but unenforceable, because the custody account is caller-substitutable	Low	Resolved
6	Pre-funding a position address permanently blocks a specific borrow	Low	Resolved
7	Doc-comments assert authorisation bindings the code does not enforce	Low	Resolved
8	A user can open the withdrawal channel but cannot close it, blocking their own exit	Low	Resolved



No	Issue Title	Severity	Status
9	Settlement events cannot express a multi-recipient distribution or be joined across the two programs	Informational	Resolved
10	Two declared error variants are dead, and each names a control that is genuinely missing	Informational	Resolved
11	Events omit the values they change, across five instruction families	Informational	Resolved
12	The close_account helper's contract is undocumented and unenforced, and its doc-comment is wrong	Informational	Resolved
13	Cross-program and cross-version bindings are enforced neither at build time nor at runtime	Informational	Resolved
14	Latent-correctness footguns that are inert today	Informational	Resolved



■ High Severity Issues

1. Un-widened u64 share arithmetic caps a repayable order at 2^{32} base units

Path zynk-orbit/src/instructions/repay.rs

Function name repay()

Description:

repay allocates each borrower's share of a repayment using arithmetic that stays entirely in u64, at programs/zynk-orbit/src/instructions/repay.rs:126-140:

```
let numerator = prepared_amount
    .checked_mul(info.remaining)
    .ok_or(ProgramError::ArithmeticOverflow)?;
let raw_share = (numerator + remaining_order - 1)
    .checked_div(remaining_order)
    .ok_or(ProgramError::ArithmeticOverflow)?;
```

Because $\text{prepared_amount} \leq \text{remaining_order}$ and the supplied positions must sum to remaining_order , the numerator approaches remaining_order^2 . That exceeds u64::MAX once the order passes 2^{32} base units, and `checked_mul` aborts the instruction. The final quotient would always fit u64; only the intermediate overflows.

Two secondary details matter. The `+` on the second line is unchecked while every operation around it uses `checked_*`, and with `overflow-checks = true` in the release profile that is a panic path rather than a wrap, it is unreachable today only because `checked_mul` fails first. And nothing bounds order size at creation: `create_order` validates only that the amount is non-zero.

No attacker is involved. This occurs during ordinary settlement.



Impact:

An order above the boundary cannot be settled through repay by anyone, including the manager. Borrowers are not left without a route, since claim computes recovery through a `min()` chain with no multiplication and remains available to an individual borrower. The loss is the protocol's orderly pro-rata settlement path: every borrower on an affected order is forced onto claim, which is the first-come-first-served race described below, and repay's all-or-nothing gate means the order cannot be settled piecewise either.

Partial repayment is a poor mitigation because transaction count scales as $R^2/2^{64}$. A 1,000,000-token order needs 27,109 transactions at 6 decimals and roughly 27.1 billion at 9 decimals, about 17 years at 50 transactions per second, which makes the principal effectively locked.

The practical ceiling is lower than the arithmetic one. Because repay must supply every open position in a single transaction, the position count is bounded around 20, giving a real-world limit of roughly 4,300 to 19,000 tokens at 6 decimals.



Recommendation:

Compute the share in u128 and check the addition:

```
let numerator = (prepared_amount as u128)
    .checked_mul(info.remaining as u128)
    .ok_or(ProgramError::ArithmeticOverflow)?;
let denom = remaining_order as u128;
let raw_share_u128 = numerator
    .checked_add(denom - 1).ok_or(ProgramError::ArithmeticOverflow)?
    .checked_div(denom).ok_or(ProgramError::ArithmeticOverflow)?;
let raw_share: u64 = raw_share_u128
    .try_into()
    .map_err(|_| ProgramError::ArithmeticOverflow)?;
```

Widening the multiplication without also moving the addition into u128 would make the unchecked + live, so both must change together.

Separately, `OrbitError::ExcessiveRepay` is declared but never raised anywhere in the workspace. It names precisely the bound this code is missing.

Status: Resolved Fixed in 5923a8f.



■ Medium Severity Issues

2. Credited amounts are taken from instruction arguments rather than a measured balance delta

Path `zynk-orbit/src/instructions/`

Function name

Description:

Every value-moving site in both programs assumes the amount transferred equals the amount requested. None calls `reload()` to measure the actual after-before difference. Affected sites include `programs/zynk-orbit/src/instructions/deposit.rs:47-51`, `programs/zynk-orbit/src/instructions/claim.rs:60-74` and `:196-200`, and `programs/zynk-core/src/instructions/replenish_and_repay.rs:36-70`.

Under a Token-2022 transfer-fee mint that assumption is false: the destination receives amount-fee while the ledger records amount.

Impact:

On deposit the user is over-credited and the vault under-funded by the fee, every cycle, cumulatively. Because the shortfall accrues in a pooled vault it is socialised across users who had no part in the transaction. On the exit side claim advances `principal_out` by the gross amount while the claimer receives less, and since claim passes `amount == repay_amount` to Core, Core forwards more than it received. There is no aggregate solvency check anywhere in either program, so the shortfall stays invisible until the last claimant cannot be paid.

Recommendation:

Credit what actually arrived:

```
let before = ctx.accounts.destination_token_account.amount;
token_interface::transfer_checked(cpi_ctx, amount,
    ctx.accounts.mint.decimals)?;
ctx.accounts.destination_token_account.reload()?;
let received = ctx.accounts.destination_token_account.amount
    .checked_sub(before).ok_or(ProgramError::ArithmeticOverflow)?;
user.principal_in = user.principal_in.checked_add(received)?;
```

Apply the same pattern in `pledge`, `borrow`, `create_order`, `replenish` and `replenish_and_repay`. If fee-bearing mints are not intended to be supported, reject them explicitly at mint admission rather than relying on the listing process to catch them.

Status: **Resolved** Fixed in 889d70f.



3. repay's all-or-nothing settlement gate is unsatisfiable for large position sets and breaks permanently on any divergence

Path zynk-orbit/src/instructions/

Function name borrow(), repay()

Description:

repay requires the supplied positions to sum exactly to the order's outstanding balance, at programs/zynk-orbit/src/instructions/repay.rs:117-120:

```
require!(open_position_total == remaining_order, OrbitError::AmountMismatch);
```

Two consequences follow.

Every open position must fit in one transaction, at three accounts each with no sharing available for ICV borrowers, which caps an order at roughly 16 positions. borrow will happily create more, since its per-position account cost is lower and allows around 24. Orders falling in that gap can never be repaid.

The equality is also evaluated against a Core-side value that only ever decreases and that no instruction in either program can raise, so once the two sides diverge the divergence is permanent.

Impact:

A liveness failure that becomes a fund lock. Co-borrowers on an affected order can never be settled through repay and must fall back on individual claim calls, which carry the race described above.

A borrower can also revert the manager's entire batch by claiming their own position to closure while the transaction is in flight, because the settlement loop has no per-entry error isolation.

Recommendation:

Relax the gate to `open_position_total <= remaining_order` with explicit handling of the remainder, so a partial settlement degrades gracefully instead of failing closed. Bound borrow's position count to what repay can actually settle in a single transaction, or the gap between the two limits will keep producing unrepayable orders.

Status: Resolved Fixed in 432bb19.



4. The withdrawal amount is u32 in a protocol whose every other principal quantity is u64

Path	programs/zynk-orbit/src/
------	--------------------------

Function name	request_withdraw()
---------------	--------------------

Description:

WithdrawRequest.amount is declared u32 at programs/zynk-orbit/src/state.rs:62-66 and widened to u64 at the point of use in programs/zynk-orbit/src/instructions/withdrawals.rs:12, :74 and :107. Every other principal quantity in the protocol, principal_in, principal_out, max_principal, amount_borrowed, is a u64.

Impact:

A single request cannot exceed 4,294,967,295 base units: 4,294.97 tokens at 6 decimals, or 4.29 at 9 decimals.

Combined with the one-live-request-per-user constraint and the admin approval each cycle requires, withdrawing 1,000,000 USDC takes 233 serialised admin round-trips, and 10,000 wSOL takes 2,329. For a protocol handling institutional principal this is a functional ceiling on the exit path rather than a theoretical one.

Recommendation:

Widen the field to u64 and adjust WithdrawRequest::INIT_SPACE accordingly.

Status:	Resolved	Fixed in e54bf71.
---------	-----------------	-------------------



■ Low Severity Issues

5. "ICV custody is consumed first" is documented but unenforceable, because the custody account is caller-substitutable

Path zynk-orbit/src/instructions/claim.rs

Function name claim()

Description:

lib.rs documents claim as "ICV custody is consumed first. Supplied positions may then recover up to the available amount from matching Core PDVs." The implementation checks only that `icv_token_account.owner == user.key()` and that its mint matches (programs/zynk-orbit/src/instructions/claim.rs:44-53, programs/zynk-orbit/src/contexts.rs:595-597). A User PDA can own arbitrarily many token accounts, and anyone can create another one for it.

Impact:

A claimer supplies a zero-balance decoy account owned by the same User PDA, making `liquid_payment` zero and forcing the entire claim onto the partner deposit vaults while their own custody remains untouched.

The documented ordering exists so a user's own funds are exhausted before partner funds are drawn. It is advisory only, and its absence directly amplifies the vault race described above.

Recommendation:

Pin the custody account to a canonical derivation, the associated token account of the User PDA for that mint, rather than accepting any account the PDA happens to own.

Status: Resolved Fixed in 5ee9406.



6. Pre-funding a position address permanently blocks a specific borrow

Path `zyng-orbit/src/instructions/borrow.rs`

Function name `borrow()`

Description:

Position accounts are created with a raw `system_instruction::create_account` at `programs/zyng-orbit/src/instructions/borrow.rs:146-164`, which fails if the target address already holds lamports. Anchor's `init` handles this case by falling back to `transfer`, `allocate` and `assign`; the hand-rolled call here has no fallback.

Anyone who can predict `(order_id, user_id)` can pre-fund that PDA and make the borrow revert with `Create Account: account ... already in use`, surfaced as `custom program error: 0x0`.

The cost is the rent-exempt minimum for a zero-data account, 890,880 lamports (roughly 0.0009 SOL). A single lamport is not sufficient, because the runtime rejects a transfer that would leave an account below rent exemption.

Impact:

Funding for one specific order and borrower can be blocked indefinitely for a fraction of a cent, by a party holding no role in the protocol. The whole transaction reverts, including position transfers that had already moved funds into the operational vault.

Reachability is limited because `order_id` is a manager-chosen 32-byte value and Solana has no public mempool. The exposure is materially larger for a partner backend that mints order ids and therefore knows them in advance, and for any retry of a failed borrow whose id has already been published on-chain.

Recommendation:

Use Anchor's `init`, which tolerates a pre-funded account, or fall back to `allocate` plus `assign` when the account already exists with zero data.

Status: Resolved Fixed in 896968e.



7. Doc-comments assert authorisation bindings the code does not enforce

Path `zynk-core/src/lib.rs`

Function name

Description:

Several doc-comments describe controls that are not implemented.

`disburse` is documented as "Only the Core-configured manager may authorize the transfer" (`programs/zynk-orbit/src/lib.rs:78-82`). That is true of the signer, but the User account supplying the destination allowlist is not seed-bound, so the payout may go to any registered user's wallet.

`execute_request` and `unpause` are documented as "Marks the timelock action as executed" (`programs/zynk-core/src/lib.rs:300-320`). No such control exists in any form, the timelock is never closed and no executed flag is set.

Three Core handlers are undocumented, and four claims across the two files misdescribe the implementation.

Impact:

Integrators and auditors read these as specifications. Each mismatch either creates a false assurance about a control that is absent, or hides a gap that would otherwise be obvious from the interface alone.

Recommendation:

Reconcile each doc-comment with the code, or implement the control it describes.

Status: Resolved Fixed in 64d8d8c & 6ff001d.



8. A user can open the withdrawal channel but cannot close it, blocking their own exit

Path Governor_v1.sol

Function name

Description:

WithdrawRequest is created with init at [b"withdraw_request", user_id] (programs/zynk-orbit/src/contexts.rs:434-456), so exactly one may exist per user. request_withdraw is unprivileged, but the only instructions that close it, approve_withdraw and reject_withdraw, are both admin-gated. There is no user-side cancel anywhere in programs/zynk-orbit/src/instructions/withdrawals.rs.

Impact:

A user who submits a request with a wrong amount or destination cannot retract it. Until an admin acts they can create no further request, so their entire withdrawal path is blocked by their own transaction, across every wallet on the account since all resolve to the same PDA.

The same property makes a stale request outlive changes to the user's wallet set.

Recommendation:

Add a cancel_withdraw instruction authorised by any wallet in signer_user.wallets, closing the request and refunding rent to the payer.

Status: **Resolved** Fixed in da18371.



■ Informational Severity Issues

9. Settlement events cannot express a multi-recipient distribution or be joined across the two programs

Path `programs/zynk-orbit/src/events.rs`

Function name

Description:

TxEvt (programs/zynk-orbit/src/events.rs:3-15) carries a single amount and a single (from, to, from_owner, to_owner) quadruple. But repay settles N positions and claim draws from ICV custody plus N partner vaults, and each emits one aggregate event (claim.rs:202-213, repay.rs:265-276, borrow.rs:188-199).

claim hard-codes order_id: [0u8; 32] even though position.order_id is in scope, and repay zeroes user_id. On the Core side, OrderReplenished (programs/zynk-core/src/events.rs:42-59) carries the vault but no user identity.

borrow emits the order total instead of the per-position amount, and puts the SPL Token program id in from_owner where the wallet belongs.

Recommendation:

Emit one event per recipient inside the settlement loops, carrying user_id, the real order_id, the position amount, the mint, and the resulting principal_out. Use the real token_account.owner rather than AccountInfo::owner for from_owner, the correct handling already exists at repay.rs:187-194.

Status: Resolved Fixed in 3d8759f.



10. Two declared error variants are dead, and each names a control that is genuinely missing

Path `programs/zynk-orbit/src/error.rs`

Function name

Description:

`OrbitError::ExcessiveRepay` (`programs/zynk-orbit/src/error.rs:41-42`) is never raised. `repay.rs:53` silently clamps an over-payment with `amount.min(remaining_order)` instead of rejecting it, which is the root of the replenishment surplus described above.

`OrbitError::CliffPeriodOver` (`:27-28`) is never raised either, and nothing anywhere checks whether a cliff has already elapsed before changing it.

Recommendation:

Raise `ExcessiveRepay` when `amount > remaining_order`, and add the elapsed-cliff check the second variant describes.

Status: Resolved Fixed in c38d93b.



11. Events omit the values they change, across five instruction families

Path `programs/zynk-orbit/src/events.rs`

Function name

Description:

AxEvent (programs/zynk-orbit/src/events.rs:17-24) has no numeric field at all, so the cliff value, the sole time lock on user custody, cannot be emitted by any handler in cliff.rs

WithdrawRequested (withdrawals.rs:45-51) omits amount, destination and mint. update_cctp_recipient in users.rs, which gates irreversible cross-chain burns, emits nothing at all. Core's Action (governance.rs:20-26) omits the value being set, and BeneficiaryAction (beneficiaries.rs:14-20) omits both the acting signer and allow_transient.

Recommendation:

Add the changed value, the acting signer and a timestamp to each event. Core's own Action event already carries signer and timestamp, so this is an omission rather than a platform limit.

Status: Resolved Fixed in 7446349.



12. The `close_account` helper's contract is undocumented and unenforced, and its doc-comment is wrong

Path `programs/zynk-core/src/utlis.rs`

Function name

Description:

The helper at `programs/zynk-core/src/utlis.rs:25-37`, duplicated verbatim at `programs/zynk-orbit/src/utlis.rs:12-26`, is byte-for-byte Anchor 0.31.1's own `common::close` and is safe as used. Account revival does not apply.

Its doc-comment claims it "zeroes out the account data to prevent reuse". Neither clause is true: `resize(0)` sets the length without a `memset`, and closing an account with deterministic seeds enables re-creation rather than preventing it.

It also declares no preconditions on `from` regarding owner, writability or type, omits the `from ==` to guard that Anchor's own `close =` constraint emits, and uses `checked_add(...).unwrap()`, which panics instead of returning an error. It is additionally exported at the crate root via `pub use`.

Recommendation:

Reserve the requested tier so the buy amount is unchanged, but defer the treasury push until the delivered count is known and refund the difference.

Status: Resolved Fixed in a241a0b.



13. Cross-program and cross-version bindings are enforced neither at build time nor at runtime

Path `programs/zynk-core/src/constants.rs`
`programs/zynk-orbit/src/utils.rs`

Function name

Description:

ZYNK_ORBIT_ID (`programs/zynk-core/src/constants.rs:5`) is the entire authentication for the capability that lets Orbit move Core's funds. It is a hardcoded constant with no compile-time link to the sibling program's `declare_id!`, and `Anchor.toml`'s `[programs.devnet]` section already declares different program ids than the source constants.

The CCTP call data (`programs/zynk-orbit/src/utils.rs:75-131`) is hand-encoded to Circle's v1 ABI and is byte-correct for v1, but nothing binds the program to that ABI: the discriminator is hashed from a runtime string literal and there is no `CCTP_PROGRAM_ID` constant anywhere in the crate.

User (`programs/zynk-orbit/src/state.rs`) carries no schema version and the program has no migration instruction, although two handlers `realloc` it in place.

Recommendation:

Assert `ZYNK_ORBIT_ID == zynk_orbit::ID` at compile time, pin the CCTP program id in a constant and check it, and add a version byte to User.

Status: Resolved Fixed in 521bc9e.



14. Latent-correctness footguns that are inert today

Path `programs/zynk-core/src/`
`programs/zynk-orbit/src/`

Function name

Description:

Several constructs are correct only by coincidence and would fail silently under an ordinary refactor. Two handlers bind `&mut` to a context field declared without `mut` (`programs/zynk-core/src/instructions/close_orders.rs:6`, `programs/zynk-orbit/src/instructions/disburse.rs:9`), so `Anchor` emits no write-back. Any future write would compile, pass review, and be discarded. One of them is `disburse`, which is precisely the handler with a known missing ledger entry.

`Config` (`programs/zynk-core/src/state.rs:21-38`) is the only hand-sized account and the only one that reallocs, so `realloc::zero = false` is safe only because `BASE_SIZE` happens to be byte-exact.

`ApproveCliffPeriod` (`programs/zynk-orbit/src/contexts.rs:527-547`) validates its request by value rather than by seed, which is sufficient only while exactly one such account can exist per user.

`#[repr(u8)]` on `UserType` (`programs/zynk-orbit/src/state.rs:4-9`) promises a wire encoding `Borsh` does not honour.

Recommendation:

Fix the `&mut` bindings, add a compile-time assertion tying `BASE_SIZE` to a derived `INIT_SPACE`, and add the seed constraint to `ApproveCliffPeriod`.

Status: Resolved Fixed in 899386e.



Security Invariants and Non-Negotiable Properties

The following security invariants were provided by the Zynk team as non-negotiable properties of the protocol. As part of this assessment, these properties were reviewed against the audited codebase to confirm that both newly introduced functionality and existing security-critical behavior remain consistent with the intended protocol design.

The validation status reflects the implementation present in the audited commit and should not be interpreted as applying to code changes made after the assessed revision.

ID	Security Invariant	Result	Assessment
INV-01	Only admin or manager may pause immediately.	Enforced	Enforced in the Pause context: authority must equal config.manager or config.admin. No other instruction writes config.paused.
INV-02	Only admin or guardian can manage beneficiary and mint whitelists.	Enforced	Enforced across whitelist_beneficiary, toggle_beneficiary, revoke_beneficiary and update_whitelisted_token_mint. The manager is excluded from all four.
INV-03	All transactions should adhere to the mint whitelisting.	Enforced	The whitelist check is present.
INV-04	An order created with a certain token can only be replenished/closed with the same token.	Enforced	Bound on the replenish and replenish_and_repay paths via <code>mint.key() == order_tracker.mint (contexts.rs:136, :211)</code> .



ID	Security Invariant	Result	Assessment
INV-05	zynk-orbit does not have any governance, it totally depends on zynk-core's config, except for the pause state. Said so, zynk-orbit doesn't consist of any pause/unpause mechanism, means LP activities need not be paused, however, the borrow/repay path gets blocked due to zynk-core's pause state.	Enforced	Structurally correct. Orbit declares no config account and no pause instruction; paused does not appear anywhere in zynk-orbit. borrow and repay are blocked indirectly through the create_order and replenish_and_repay CPIs. Note the unpaused surface also includes disburse, cctp and the ICV branch of claim, which are not LP activities.



ID	Security Invariant	Result	Assessment
INV-06	For Core transactions, amount_in must be greater than equal to amount_out.	Enforced	Closure is guarded by amount_in >= amount_out in replenish (replenish.rs:44, :50), record_order (:32) and replenish_and_repay (:71).
INV-07	A single signer (manager/admin/guardian) cannot perform more than one governance operation.	Enforced	Separation of duty enforced via req_by != authority in ack_timelock (governance.rs:54), revoke_timelock (:35) and execute_request (:76, :77).
INV-08	Guardian update strictly requests both the timelock to be elapsed and the consent of the current Guardian.	Enforced	UpdateGuardian requires eta_ready && acked (governance.rs:82-86), the only action requiring the conjunction. Across every reachable requester/acker/executor combination the guardian participates as either acker or executor. Operational note: the update must be requested by the manager, since an admin-requested one cannot be executed or revoked by anyone.
INV-09	Users (as in LP/ICV/NCW) can interact with the protocol only using the whitelisted wallets/accounts (max 3).	Enforced	The max of 3 is structural (wallets: [Pubkey; 3]). Membership is checked in deposit, claim, request_withdraw, approve_cliff_period, reject_cliff_period, repay and disburse. Note approve_withdraw validates the destination against the stored request rather than the live wallet list, which update_wallets can change in between.



ID	Security Invariant	Result	Assessment
INV-10	Orbit user's net principal (principal_in - principal_out) must never exceed stored max_principal.	Enforced	Implemented
INV-11	If whitelisted_partners is non-empty, a borrow can only happen for approved partners (extracted numeric code from partner_id). If empty, all partners are permitted.	Enforced	Implemented exactly as specified at borrow.rs:60-66. Two robustness notes: the user account is read from remaining_accounts with no ownership check or PDA re-derivation, unlike repay (repay.rs:78-83); and extract_partner_number accepts a leading '+' and leading zeros, so zp_+12345 and zp_012345 resolve to the same allowlist entry while hashing to different Core partner identities.
INV-12	For CCTP, funds cannot flow from the user PDAs to any other addresses not whitelisted in their cctp_recipients list.	Enforced	Enforced for ICV transfers at cctp.rs:42-53 against user.cctp_recipients, alongside the cliff check at cctp.rs:37-40.
INV-13	Core orders cannot be created for any other beneficiaries not whitelisted (involves partner mapping).	Enforced	The beneficiary PDA is re-derived from partner_id and the destination token account owner, with is_active and public_key checks (contexts.rs:62-68). The partner mapping is enforced by the address derivation itself, so a beneficiary whitelisted for one partner cannot be used for another.



ID	Security Invariant	Result	Assessment
INV-14	Borrow from several Orbit users is permissible (total number of PDAs is subject to the transaction size limit). Each borrow can create multiple position PDAs, each for the ICVs involved - which define the share of that specific ICV.	Enforced	The per-position loop (borrow.rs:42-200) creates a distinct Position PDA per user recording that user's share via amount_borrowed. Operational note: the account limit binds near 12 positions on borrow, but repay requires every open position in one call and caps nearer 16, so an order exceeding that cannot be settled through repay.
INV-15	If multiple ICVs do not suffice the required amount to create an order, the remaining must be facilitated from the ZOV itself. Hence, the borrow shares must take this into consideration.	Enforced	ICV always retrieve their full funds either via claim or via repay or combination of both
INV-16	Only Orbit can invoke the replenish_and_repay function in Core.	Enforced	The orbit_authority account is required as a signer with seeds = [ORBIT_CPI_AUTHORITY_SEED] and seeds::program = ZYNK_ORBIT_ID (contexts.rs:152-159). Only the Orbit program can produce that signature. Note ZYNK_ORBIT_ID is a compile-time constant currently holding the localnet id while Anchor.toml declares a different devnet id.



ID	Security Invariant	Result	Assessment
INV-17	Transient pull_and_create_orders are permitted, irrespective of the transient_allowed flag for beneficiaries.	Enforced	pull_and_create_order.rs:88-101 handles the transient/persistent split with no reference to allow_transient anywhere. create_order (same context, sibling instruction) keeps its own separate gate at create_order.rs:58 untouched
INV-18	On Solana- Funds cannot move out of ICV without an order that comes from an allowed partner. For CCTP flows - funds can only move to a whitelisted beneficiary (a vault dedicated to this ICV's allowed partners once the EVM contract goes live; verifiable separately on EVM against the declared partners).	Enforced	On-Solana half: borrow checks correctly - extract_partner_number then user.whitelisted_partners.contains(...) (borrow.rs:27,61-66) before any funds move. CCTP half: the per-user allowlist is real and enforced



Functional Tests

Zynk-core

initialize

- ✓ Should fail to initialize with empty whitelisted token mints vector
- ✓ Should fail to initialize with invalid token mint address in vector
- ✓ Should fail to initialize with duplicate token mints in vector
- ✓ Initializes the protocol with multiple token addresses

Order

- ✓ Should fail creating transient order when transient disabled
- ✓ Should be able to create transient pull order, even when transient disabled
- ✓ Can replenish tokens multiple times before order is closed
- ✓ Should fail when wrong partner deposit vault is used to replenish
- ✓ Should be able close the order by manager via replenish
- ✓ Should fail when non-manager tries to close order
- ✓ Should fail when trying to close an already closed order via replenish
- ✓ Should fail when trying to replenish a closed order
- ✓ Should fail when partner deposit vault has insufficient balance
- ✓ User can create and replenish/close order with same mint token
- ✓ Should fail to close order created with first token using second token mint
- ✓ User is able to pull and create order and close order with same mint token
- ✓ Should fail to close pull order with a different mint token than the order was created with



governance

- ✓ Should be able to pause by manager
- ✓ Should be able to pause by admin
- ✓ Should not be able to pause by non-authority
- ✓ Should not be able to request timelock by unauthorized authority
- ✓ Should be able to request timelock by manager
- ✓ Should not be able to execute timelock before eta or ack
- ✓ Should not be able to ack timelock by requester or unauthorized authority
- ✓ Should not be able to execute unpause using a wrong timelock request

beneficiaries (whitelist_beneficiary / toggle_beneficiary / revoke_beneficiary)

- ✓ Should whitelist partnerOperationalWallet as beneficiary - transient disabled
- ✓ Should temporarily disable whitelisted partnerOperationalWallet as beneficiary
- ✓ Should revoke whitelisted partnerOperationalWallet as beneficiary
- ✓ Should whitelist partnerOperationalWallet as beneficiary - transient enabled

token_whitelist

- ✓ Should successfully add a new token mint to the whitelist by admin
- ✓ Should successfully add a new token mint to the whitelist by guardian
- ✓ Should fail when adding an already whitelisted token mint
- ✓ Should fail when adding with invalid/null address
- ✓ Should fail when unauthorized signer tries to add token mint
- ✓ Should successfully remove a token mint from the whitelist
- ✓ Should fail when removing a token mint that is not whitelisted
- ✓ Should fail when unauthorized signer tries to remove token mint



zynk-orbit

deposit

- ✓ Should not be able to deposit funds into the contract for NCW user
- ✓ Should not be able to deposit by non-whitelisted user
- ✓ Should not be able to exceed deposit more than max_deposit
- ✓ Should be able to deposit funds into the contract for ICV user
- ✓ Should be able to deposit funds into the contract for LP user
- ✓ ICV user should deposit funds into the contract

borrow

- ✓ Should not be able to borrow against LP user
- ✓ Should not be able to borrow if partnerId is not whitelisted for all of the position user (Open user are allowed)
- ✓ Should be able to borrow against the ICV user
- ✓ Should be able to borrow against NCW user
- ✓ Should be able to borrow with multiple positions (3 positions) for an order with different users
- ✓ Manager should borrow (open position) against the ICV user

repay

- ✓ Manager repays against the ICV user (full repay)
- ✓ Manager repays against the NCW user (full repay)
- ✓ Should be able to repay with multiple positions (3 positions) for an order
- ✓ Should be able to do partial repay and each position should be dissolved proportionally
- ✓ Should not be able to repay if sum of amount of all positions is not equal to remaining amount in order tracker
- ✓ Should not be able to repay using different mint token than that of borrow
- ✓ Should not close position to a different user than from which funds were borrowed
- ✓ Should repay only open principal and retain excess replenishment in ZOV
- ✓ Manager should fully repay the position against the ICV user



claim

- ✓ Should not be able to claim before the cliff period is over
- ✓ Should not allow an NCW claim without claimable principal
- ✓ Should not be able to claim amount more than that of the current balance
- ✓ Should be able to claim after the cliff period is over
- ✓ Should claim liquid ICV principal plus available PDV recovery
- ✓ Should not claim unaccounted token balances after revoke and rewhitelist



withdrawals

- ✓ NCW user should NOT be able to raise a withdraw request
- ✓ Should NOT raise a withdraw request if amount is greater than balance
- ✓ No user other than the primary account holder should be able to raise a withdraw request
- ✓ ICV user should raise a partial withdrawal request
- ✓ Should NOT be able to raise a withdraw request if one already exists
- ✓ LP user should be able to raise a withdraw request
- ✓ Non-admin wallet should NOT be able to approve a withdraw request
- ✓ Should NOT approve a withdraw request if source token balance is less than the withdraw amount
- ✓ Admin should approve the ICV withdrawal request
- ✓ Primary account holder should be able to reject (cancel) their own withdraw request
- ✓ Non-primary-account wallet should NOT be able to reject (cancel) a withdraw request

revoke

- ✓ Admin should revoke the ICV user whitelist
- ✓ Should be able to revoke whitelist for NCW user
- ✓ Should be able to revoke whitelist for LP user
- ✓ Should not be able to revoke by non admin wallet

cctp

- ✓ Should fail CCTP from ovault with zero amount
- ✓ Should fail CCTP from ovault with unauthorized signer
- ✓ Should fail CCTP from user before cliff period is over
- ✓ Should fail CCTP from user by non-manager signer
- ✓ Should fail CCTP from user with invalid token mint
- ✓ Should allow manager to invoke CCTP from ICV user
- ✓ Should allow manager to initiate CCTP from ovault
- ✓ Should allow manager to initiate CCTP from spender



Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of Zynk. We performed our audit according to the procedure described above. In the end Zynk Labs team Fixed all Issues.



Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

September 2026

For

Zynk



Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com